

EX 1 THREE-LAYER NEURAL NETWORK FROM SCRATCH

DATE:

Problem Statement:

Design and implement a three-layer neural network from scratch using Python. Train the network using the backpropagation algorithm with appropriate activation and loss functions. Apply the model to recognize handwritten digits using the MNIST dataset.

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

np.random.seed(42)

# Load MNIST
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Use a smaller subset so it runs faster on a normal laptop
X_train = X_train[:10000]
y_train = y_train[:10000]

# Normalize and flatten
X_train = X_train.reshape(-1, 784) / 255.0
X_test = X_test.reshape(-1, 784) / 255.0

# One-hot encoding without sklearn version problems
def one_hot(y, classes=10):
    result = np.zeros((len(y), classes))
    result[np.arange(len(y)), y] = 1
```

```
return result
```

```
y_train_encoded = one_hot(y_train)
```

```
# Network architecture
```

```
input_size = 784
```

```
hidden_size = 64
```

```
output_size = 10
```

```
W1 = np.random.randn(input_size, hidden_size) * 0.01
```

```
b1 = np.zeros((1, hidden_size))
```

```
W2 = np.random.randn(hidden_size, output_size) * 0.01
```

```
b2 = np.zeros((1, output_size))
```

```
# Sigmoid
```

```
def sigmoid(x):
```

```
    x = np.clip(x, -500, 500)
```

```
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):
```

```
    return x * (1 - x)
```

```
# Training
```

```
epochs = 10
```

```
learning_rate = 0.1
```

```
losses = []
```

```
for epoch in range(epochs):
```

```
    total_loss = 0
```

```
    for i in range(len(X_train)):
```

```
        x = X_train[i:i+1]
```

```
        y = y_train_encoded[i:i+1]
```

```
        # Forward propagation
```

```
        z1 = np.dot(x, W1) + b1
```

```
        a1 = sigmoid(z1)
```

```
        z2 = np.dot(a1, W2) + b2
```

```
        a2 = sigmoid(z2)
```

```
        # Cross entropy loss
```

```
        loss = -np.mean(y * np.log(a2 + 1e-8))
```

```
        total_loss += loss
```

```
        # Backpropagation
```

```
        dz2 = a2 - y
```

```
        dW2 = np.dot(a1.T, dz2)
```

```
        db2 = dz2
```

```
        dz1 = np.dot(dz2, W2.T) * sigmoid_derivative(a1)
```

```
        dW1 = np.dot(x.T, dz1)
```

```
        db1 = dz1
```

```

# Update weights
W2 -= learning_rate * dW2
b2 -= learning_rate * db2

W1 -= learning_rate * dW1
b1 -= learning_rate * db1

average_loss = total_loss / len(X_train)
losses.append(average_loss)

print("Epoch", epoch + 1, "Loss:", round(average_loss, 4))

# Plot training loss
plt.figure(figsize=(7, 5))
plt.plot(range(1, epochs + 1), losses, marker='o')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss")
plt.grid()
plt.show()

# Prediction
def predict(image):
    image = image.reshape(1, 784) / 255.0

    hidden = sigmoid(np.dot(image, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

```

```
return np.argmax(output)
```

```
# Test sample
```

```
index = 100
```

```
prediction = predict(X_test[index])
```

```
plt.figure(figsize=(4, 4))
```

```
plt.imshow(X_test[index].reshape(28, 28), cmap="gray")
```

```
plt.title("Actual: {} | Predicted: {}".format(y_test[index], prediction))
```

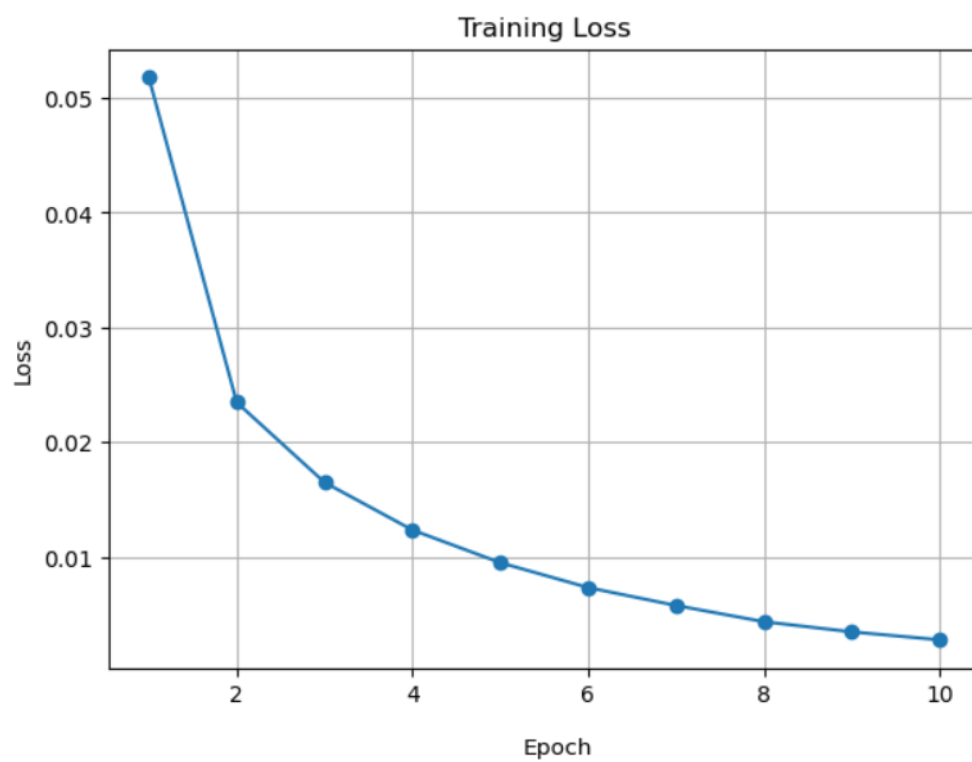
```
plt.axis("off")
```

```
plt.show()
```

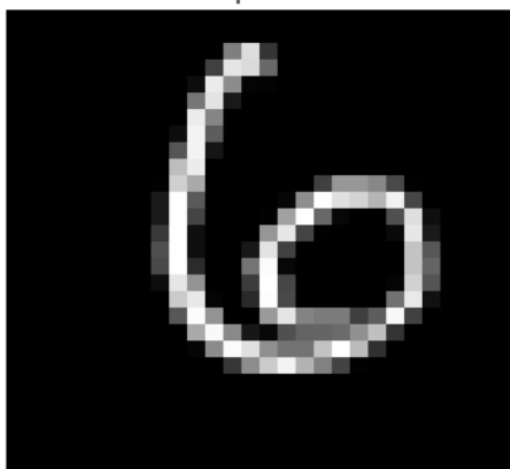
```
print("Actual Digit :", y_test[index])
```

```
print("Predicted Digit :", prediction)
```

Epoch 1 Loss: 0.0518
Epoch 2 Loss: 0.0235
Epoch 3 Loss: 0.0165
Epoch 4 Loss: 0.0123
Epoch 5 Loss: 0.0095
Epoch 6 Loss: 0.0073
Epoch 7 Loss: 0.0057
Epoch 8 Loss: 0.0043
Epoch 9 Loss: 0.0034
Epoch 10 Loss: 0.0028



Actual: 6 | Predicted: 5



Actual Digit : 6
Predicted Digit : 5